Observe, Expand, Recover: An Observability-Guided Agent for Safe Query Expansion

An Honest Account of What Runtime Telemetry Can and Cannot Buy a Query Expansion Agent

Author One*, Author Two†
*Affiliation One, email@example.com
†Affiliation Two (AI Observability), email2@example.com

***Abstract*—Large language model (LLM) query expansion is typically applied either unconditionally or not at all, despite well-documented evidence that unconditional expansion is net-harmful on a non-trivial fraction of queries. We formulate query expansion instead as a bounded, observable agentic decision: an agent observes retrieval-health telemetry, selects a corpus-grounded expansion operator, executes it, observes the consequences through runtime telemetry, and decides to accept, roll back, replan, or abstain — a closed loop directly analogous to site-reliability engineering's observe-diagnose-act discipline, applied to a retrieval pipeline instead of a production service. We study three research questions on TripClick, a real long-tail health search benchmark with genuine head/torso/tail query-frequency structure: (RQ1) can qrel-free retrieval and agent telemetry predict whether an expansion will help or hurt; (RQ2) does a closed-loop agent conditioned on that telemetry outperform an otherwise identical static, failure-conditioned expansion pipeline; and (RQ3) can observability-driven recovery detect and correct large, deliberately injected failures even when it cannot reliably judge subtle, naturally occurring ones. We report all three findings honestly, including negative and null results. Qrel-free telemetry carries a real but weak signal for predicting expansion outcomes (Pearson $r \approx 0.13$–0.15 against true nDCG@10 change), a ceiling that persists across three feature families (lexical, lexical plus cross-encoder reranker scores, and lexical plus dense embedding similarity) and two model classes (linear and gradient-boosted). On natural queries at full test-set scale ($n = 1175$), the closed-loop agent does *not* significantly outperform a static comparator built from identical components (mean nDCG@10 difference -0.0007, 95% CI $[-0.0024, 0.0007]$) and costs 27% more model calls for no measurable quality gain. Under deliberately injected content-level corruption, however, the same weak verifier shows a directionally positive recovery effect, while remaining blind to system-level degradation whose signal lies outside its feature family. We report a genuine implementation defect discovered mid-study — a cross-encoder reranker computed but never applied to the ranking it was meant to refine — and its correction, as part of the paper's engineering record rather than a footnote, since fixing it materially changed our own results. We argue that these findings, taken together, constitute a precise map of where observability-guided control earns its keep in a retrieval agent and where it does not, and that this map is itself the paper's contribution.**

***Index Terms*—query expansion, information retrieval, agentic systems, AI observability, retrieval-augmented generation, large language models, site reliability engineering**

I. Introduction

Large language models can generate useful pseudo-documents, alternative phrasings, and disambiguating context for a search query [1], [2], and corpus-grounded generation — conditioning the rewrite on retrieved evidence rather than the query text alone — improves on naive query-only rewriting. Existing work has established that LLM query expansion *can* help. It has not established when an operational system should trust an expansion enough to use it. In practice, expansion is applied unconditionally to every query or not at all; neither policy is safe, since unconditional rewriting is reliably net-harmful on a measurable fraction of queries, particularly underspecified or already-well-served ones.

We treat this as a control problem rather than a generation problem. Borrowing the observe-diagnose-act-recover discipline of site reliability engineering [3] and the trace/metric vocabulary of OpenTelemetry [4], [5], we build an agent whose action space is a small set of interpretable, corpus-grounded expansion operators, whose control loop alternates actions with structured observations in the manner of ReAct [6], and whose accept, roll-back, replan, and abstain decisions are driven by runtime telemetry of the retrieval episode rather than by the language model's own self-assessment. The central empirical question is not whether an LLM can produce a useful expansion — that is established — but whether *runtime observability* can make expansion selective, recoverable, and cost-aware.

This paper reports what we found when we tried to answer that question rigorously, including where the answer was no. We see this as a necessary complement to the (mostly positive) existing literature on LLM query expansion: understanding the boundary of when an agentic, observability-driven control layer helps is only possible by also reporting the conditions under which it does not.

A. Research Questions

RQ1 (Detection). Can qrel-free retrieval and agent-runtime telemetry identify unhealthy retrieval states and predict whether query expansion is likely to help or harm, without access to relevance judgments at inference time?

RQ2 (Control). Does an agent that conditions expansion actions on that telemetry outperform an otherwise identical *static*, failure-conditioned expansion pipeline — the same retriever, grounding mechanism, generator, and fusion method, differing only in whether a closed control loop can observe an action's consequences and recover from a bad one?

RQ3 (Recovery). After a harmful rewrite or a simulated system-level fault, can observability-driven verification detect the degradation and select an appropriate recovery action to restore retrieval quality, under a bounded action budget — and does this hold even for a verifier whose general-purpose detection power (RQ1) is only weakly informative?

B. Contributions

We contribute (1) an agent architecture that treats LLM query expansion as a bounded, typed-tool action inside a closed observability loop, rather than as an unconstrained generation step; (2) a telemetry model spanning retrieval-, agent-, and system-level signals, together with a compact failure taxonomy that gives the observability layer something concrete to predict and recover from; (3) a rigorous, qrel-free verifier calibration methodology validated against real relevance judgments, including a systematic search across feature families and model classes to characterize the ceiling of qrel-free detection on this task; (4) a full-scale (TripClick TAIL test set, $n = 1175$) evaluation of the closed-loop agent against a matched static comparator; (5) a fault injection protocol and recovery study isolating whether observability value survives even when general detection is weak; and (6) an honest account of a real implementation defect — discovered, diagnosed, and corrected during the study — that materially changed our reported recovery results, offered as a methodological contribution in its own right.

II. Related Work

LLM query expansion. Query2doc [1] generates pseudo-documents from an LLM and appends them to the query for sparse or dense retrieval. MuGI [2] generates and fuses multiple expansions rather than one. Both treat expansion as an unconditional, one-shot text-generation step; neither conditions the decision to expand on retrieval-state telemetry, and neither provides a mechanism to detect or recover from a harmful rewrite once issued.

Agentic control loops. ReAct [6] interleaves reasoning traces with actions and environment observations, and is the template for our own observe-act-observe structure, but at a much smaller and more measurable action space: our agent selects among a fixed set of typed retrieval operators rather than free-form tool calls, which keeps the resulting policy interpretable and lets a lightweight, non-LLM controller drive it.

Observability and reliability engineering. OpenTelemetry's separation of traces, metrics, and logs [4], and its GenAI-specific semantic conventions [5], supply a naming discipline for the telemetry we collect, without requiring a commercial observability backend for the experiment itself. Google's SRE framing of a Service Level Objective as a target range on a measurable indicator, rather than a single metric to optimize [3], motivates our Agent Search SLO: maximize retrieval quality subject to explicit harm, cost, and latency constraints frozen *before* evaluation, rather than tuned to the test set post hoc.

Benchmarks. TripClick [7] is a real health search log with genuine head/torso/tail query-frequency structure, in contrast to benchmarks that treat "hard" queries as simply low-BM25-score ones. MS MARCO Chameleons [8] isolates queries that resist conventional reformulation; TREC DL 2019 [9] supplies deep, graded NIST judgments; BEIR's NFCorpus [10] is a small, fast benchmark used here purely for engineering iteration, not for any reported finding.

III. System Design

A. Agent State and Control Loop

The agent's state at step t is $s_t = (q_0, q_t, R_t, O_t, H_t, B_t)$, where q_0 is the immutable original query, q_t the active search expression, R_t the current ranking, O_t the observability state, H_t the episode's action/observation history, and B_t the remaining action budget. The control loop is

$$\textsc{Observe} \rightarrow \textsc{Diagnose} \rightarrow \textsc{Act} \rightarrow \textsc{Retrieve} \rightarrow \textsc{Observe} \rightarrow \textsc{Verify}$$

The critical property this loop is built to test is that the second decision (accept/rollback/replan/stop) depends on evidence the agent's own first action created — as opposed to a static pipeline that receives the same query, initial ranking, evidence, generator, and verifier, but executes exactly one predetermined route. Isolating this dependency, not simply comparing against unmodified BM25, is the paper's central experimental contrast (Section VI).

The action space is deliberately small and interpretable: $A = \{\textsc{NoExpand}, \textsc{Vocabulary}, \textsc{Context}, \textsc{Ambiguity}, \textsc{EntityNormali}$ $\textsc{Vocabulary}$ maps informal user language onto corpus-compatible terminology; $\textsc{Context}$ adds a corpus-supported missing facet to an underspecified query; $\textsc{Ambiguity}$ disambiguates toward the interpretation the evidence best supports; $\textsc{EntityNormalize}$ resolves aliases, abbreviations, and canonical-name mismatches. The LLM is invoked through exactly one typed tool, `generate_expansion(action, query, evidence)`; it is never the sole arbiter of whether its own output is used. That judgment is made by a separate, non-generative controller (Section III-D).

B. Observability Model

Every query episode produces one trace with spans `retrieve_original` → `build_evidence` → `plan_action` → `generate_expansion` → `retrieve_expanded` → `rerank` → `verify` → `recover_or_accept` → `fuse`. The observation vector O_t spans three telemetry families. *Retrieval telemetry*: top score, score margin, normalized score entropy, query-term coverage, mean rare-term IDF, entity overlap, top-k ranking overlap against the previous state, reranker/BM25 disagreement, lexical result coherence, and ranking stability

under a small controlled query perturbation. *Agent telemetry*: selected operator, trajectory length, previous verifier score, rewrite-to-original semantic drift, introduced entities, repeated actions, and remaining budget. *System telemetry*: per-tool latency, token counts, and tool errors or timeouts. Every feature here is computable without relevance judgments, which is what makes the resulting verifier usable at inference time.

C. Failure Taxonomy

We label episodes into ten failure categories — vocabulary mismatch, underspecification, ambiguity, entity mismatch, intent drift, confirmation bias, retriever disagreement, tool degradation, budget exhaustion, and healthy retrieval — each with an observable telemetry signature and a preferred recovery action. This taxonomy gives the observability layer something concrete to predict and recover from, rather than treating "observability" as post hoc dashboards around a search experiment; it directly motivates the fault-injection protocol of Section VII.

D. Verifier and Controller

The verifier, `verify(previous_state, new_state)`, produces a scalar judgment of whether an action's consequences look like an improvement, using telemetry alone. Rather than hand-picking its weights, we calibrate them against ground truth: for a sample of episodes, we execute one expansion action, compute the true nDCG@10 change of the resulting ranking against real relevance judgments, and fit the verifier's weights to predict that change from the telemetry features described in Section III-B. Ground truth is available only during this offline calibration; the deployed verifier never sees a relevance judgment. We treat this fitted model as an artifact — trained by a dedicated script and loaded at inference time — rather than a set of constants transcribed from a one-off calibration run, so that recalibration is a script re-run, not a source edit. The recovery controller is a small deterministic finite-state machine: below a calibrated harmful threshold the agent rolls back to the pre-action ranking; above a calibrated accept threshold it fuses the new ranking in via reciprocal-rank fusion; in between it replans with an untried operator, subject to a hard cap of two model calls per query. Both thresholds are calibrated empirically (Section IV), not guessed.

IV. Experimental Setup

A. Datasets

TripClick [7] is the primary benchmark: 1,523,878 documents and a 3,525-query test set split evenly across head, torso, and tail frequency buckets (1,175 queries each), the only one of our candidate datasets that reflects genuine observed query-frequency structure rather than treating "tail" as a synonym for "hard for BM25." All reported results in this paper concern the TAIL split, since it is the frequency regime the paper's motivating claim is about. NFCorpus [10] (3,633 documents, 323 queries) was used exclusively for engineering iteration and does not contribute to any reported finding.

B. Implementation

Sparse retrieval uses BM25 over a SQLite FTS5 index rather than a JVM Lucene index; this choice was forced by a JVM crash under Pyserini on our target hardware and, in a corpus of this scale, surfaced two further implementation lessons worth recording. First, FTS5's default combining operator between space-separated MATCH tokens is *conjunctive*, not disjunctive: naively passed multi-term queries silently required every term to co-occur in a document, collapsing recall on any multi-term query without raising an error. Second, FTS5 indexes only the column used in `MATCH`; any lookup on a companion column such as a document identifier degrades to a full table scan, invisible at small scale and catastrophic at 1.5M rows (a single telemetry computation went from 29.2 s to 1.65 s, an $18\times$ speedup, once a companion indexed table was added for identifier lookups). We report both because they are the kind of defect that silently corrupts results rather than crashing, and are easy to miss without an independent sanity check against a published baseline range. Cross-encoder reranking (`cross-encoder/ms-marco-MiniLM-L6-v2` [11]) is applied to the top 100 BM25 candidates, reranking the top 50. Generation uses a locally hosted, 4-bit quantized 2–3B-parameter model with a hard cap of two calls per query episode and a 32–64 token structured output, so that the language model is never the entire decision surface, only one typed tool within it.

C. Verifier Calibration Methodology

We calibrate against $n = 1168$ TripClick TAIL episodes (one expansion action per query, real nDCG@10 ground truth), and test three feature families against two model classes each to characterize where the detection ceiling actually lies, rather than accepting the first plausible fit. The families are: (i) six retrieval-telemetry deltas (score margin, coverage, coherence, stability, drift, reranker disagreement); (ii) the same six plus four reranker- and length-derived features (top-k overlap, reranker top score, reranker score margin, query length ratio); and (iii) family (ii) plus three dense-embedding features (query rewrite embedding drift, cross-document embedding coherence, and expansion-to-evidence embedding groundedness, using a locally hosted embedding model). We compare a fixed-weight hand-calibrated baseline, a 5-fold cross-validated linear fit, and a 5-fold cross-validated shallow gradient-boosted fit, on each family, against true nDCG@10 change under Pearson correlation. Thresholds for the recovery controller are calibrated by checking, in the same calibration data, which candidate cutoffs actually separate true-harmed episodes from the rest — not by inspecting score percentiles blind to outcome.

D. Metrics

Retrieval quality is nDCG@10, Recall@100, and MRR. Because mean nDCG can conceal query-level regressions, we additionally report the fraction of queries helped, unchanged, and harmed by an intervention relative to a baseline. For the recovery study (Section VII) we report Detection Recall

TABLE I
RQ1: QREL-FREE DETECTION CORRELATION ($n = 1168$ TRIPCLICK TAIL EPISODES UNLESS NOTED). FIXED-WEIGHT IS A HAND-CALIBRATED BASELINE; LINEAR AND GBDT ARE 5-FOLD CROSS-VALIDATED FITS ON THE SAME FEATURES.

Feature family	Fixed-weight	Linear (CV)	GBDT (CV)
Lexical (6 features)	0.127	0.148	0.161
+ reranker/length (10)	−0.064	0.134	0.045
+ dense embedding (13)[1]	—	0.094	—

TABLE II
RQ2: FULL TRIPCLICK TAIL TEST SET ($n = 1175$). STATIC QE AND AGENT SHARE THE SAME RETRIEVER, GROUNDING, GENERATOR, VERIFIER, AND FUSION MECHANISM; AGENT ADDITIONALLY HAS A CLOSED OBSERVE-VERIFY-RECOVER LOOP.

System	nDCG@10	Recall@100	MRR
BM25 only	0.2279	0.6462	0.2254
Static QE	0.2279	0.6481	0.2262
Agent	0.2272	0.6476	0.2254

$P(\text{intervened} \mid \text{harmful})$, False Alarm Rate $P(\text{intervened} \mid \text{healthy})$, and the recovery gain in mean nDCG@10 with versus without observability-driven recovery on identically corrupted episodes. Significance is assessed via paired bootstrap (10,000 resamples) throughout.

V. RQ1: DETECTION

Table I reports Pearson correlation between each fitted verifier and true nDCG@10 change, on held-out folds, for each of the three feature families described in Section IV-C.

Three findings emerge. First, no fitted model, on any feature family, clears a Pearson correlation of 0.17: the ceiling is not a modeling artifact of one particular choice. Second, richer features do not help: adding reranker- and embedding-derived signals never produces a statistically decisive improvement over the plain lexical family, and the fitted embedding-augmented model (0.094) does not even recover the lexical family's own full-scale linear result (0.134). Third, model class does not obviously matter either — GBDT outperforms linear on the 6-feature family but underperforms it on the 10-feature family, consistent with noise rather than a genuine nonlinear signal at this sample size. We interpret this as a real, if unwelcome, empirical ceiling: with this telemetry vocabulary, on this corpus, qrel-free detection of whether an LLM query rewrite helped or hurt carries real but weak signal, on the order of $R^2 \approx 1$–3% of outcome variance explained.

VI. RQ2: CONTROL

Both expansion pipelines are statistically indistinguishable from plain BM25 at this scale. Comparing Agent against Static QE specifically — the paper's central contrast, since both share every component except the closed control loop — the agent changes the outcome on only 1.4% of queries (0.5% helped, 0.9% harmed relative to static), and the paired-bootstrap mean nDCG@10 difference is -0.0007 (95% CI $[-0.0024, 0.0007]$, not significant). The agent additionally costs 27% more model calls (1.27 vs. 0.99 per query) and 45% higher mean latency (16.2 s vs. 11.1 s) than the static pipeline, for no offsetting quality gain. Tracing this to its source: the recovery controller's REPLAN action, whose entire purpose is to let the closed loop add value beyond a single-shot accept/reject gate, is exercised on only about 1% of episodes when thresholds are calibrated from raw score percentiles, rising to roughly 16% once thresholds are instead calibrated against real outcomes (Section IV-C) — and even then, the additional attempts this buys are, per Section V, close to a coin flip on whether they help or hurt, netting to no aggregate gain at real added cost. **RQ2's answer is no**: on natural queries, an observability-guided closed loop built on this verifier does not outperform a matched static pipeline.

VII. RQ3: RECOVERY

RQ1 and RQ2 leave open whether observability has *any* practical value here, or whether a weak general detector is simply useless. We test a narrower, more targeted claim: that the same weak verifier can still serve as a safety net against *large, obvious* failures, even though it cannot reliably judge *subtle, natural* ones. We inject three fault types into the live pipeline at their natural point of corruption — an unsupported entity appended to the generated expansion, a grounding passage swapped for a topically similar but non-supporting one before generation, and the reranker disabled as a tool-level fault — and compare, on the identical corrupted episode, an ablation controller that always accepts against the real calibrated controller.

A. A Genuine Implementation Defect, Found and Corrected

An early result on this study looked strong: entity-injection recall 1.000, grounding-corruption recall 0.773, with recovery gains an order of magnitude larger than anything observed under RQ2. But the third fault type, disabling the reranker, showed almost no effect (recall 0.185), which prompted an investigation into why. Tracing the reranked ranking through the codebase revealed that its output was computed — at real latency cost — and fed into telemetry, but never actually applied to the ranking the agent accepted, fused, or evaluated; the accepted ranking was unconditionally the raw BM25 order, regardless of whether reranking succeeded. Disabling the reranker was, mechanically, a no-op, which fully explains why that fault type alone was undetectable regardless of verifier quality.

We corrected this: the reranked top-k now leads the accepted ranking when reranking succeeds, with remaining BM25-order candidates appended to preserve full recall depth. This fix, on its own, made the other two fault types' results substantially *worse* (grounding-corruption recovery gain flipped from $+0.028$ to -0.012), which on inspection traced to a second, subtler defect the first fix exposed: the verifier's

[1]Embedding family tested at $n = 368$ after an initial $n = 123$ attempt proved confounded by sample size; the 10-feature lexical family's own linear fit on that same reduced sample scored only 0.070, versus 0.134 at full scale, illustrating how noisy this correlation regime is below a few hundred episodes.

TABLE III
RQ3: FAULT INJECTION RECOVERY STUDY, CORRECTED PIPELINE, TRIPCLICK TAIL. GAIN IS MEAN NDCG@10 WITH MINUS WITHOUT OBSERVABILITY-DRIVEN RECOVERY ON IDENTICALLY CORRUPTED EPISODES; CI FROM 10,000-RESAMPLE PAIRED BOOTSTRAP.

Fault type	Recall	False Alarm	Gain (95% CI)
Entity injection	1.000	0.992	$+0.0068$ NS
Grounding corruption	0.600	0.550	-0.0019 NS
Reranker disabled	0.185	0.115	$+0.0038$*

NS: not significant, CI straddles zero. *: significant but an order of magnitude smaller than the pre-correction estimate.

telemetry was still computed from the pre-rerank ranking, so it was now judging a different top-k than the one actually being accepted. We corrected this too, by reordering the same BM25-scored candidates to match the accepted ranking before computing telemetry, keeping every score-based feature on one homogeneous scale while reflecting the true accepted order. We report this sequence in full because it materially changed our own conclusions, and because we believe silently editing it out of the historical record would understate a real risk in this class of system: a component computed for observability purposes alone is easy to leave disconnected from the outcome it is meant to explain, and that disconnection can look, from the metrics alone, like a genuine finding.

B. Validating the Correction

Following the correction, we re-ran the full verifier calibration ($n = 1168$) on the corrected pipeline to check whether the deployed verifier needed retraining. It did not: the existing model scored $r = 0.146$ against ground truth generated by the corrected pipeline — marginally *better* than its own original calibration score of 0.134 — while a fresh linear refit on the corrected data scored only 0.112 and a fresh GBDT refit collapsed to 0.001, neither clearing the improvement bar needed to justify replacing the deployed model. The existing recovery thresholds were similarly still well-placed: the corrected score distribution's median (-0.036) and 25th percentile (-0.059) land almost exactly on the deployed accept and harmful cutoffs. We take this as evidence that the defect corrections were themselves correct — the resulting telemetry is consistent enough with the original calibration's semantics that no compensating retraining was required — while still materially changing the recovery study's own numbers, since those depend on the ranking each fault type actually produces, not on the verifier's coefficients.

C. Corrected Results

[This table and the surrounding discussion report the $n = 149$ validation sample used to confirm the defect correction. A full test-set-scale ($n = 1175$) replication was in progress at the time of writing and will be finalized before submission; see Section IX.]

At $n = 149$, both content-corruption fault types show directionally positive recovery gains, consistent with the pre-correction result's qualitative shape, but neither reaches statistical significance once the pipeline defect is corrected. The reranker-disabled fault type's small effect is mechanically unaffected by the correction, as expected, since that condition explicitly bypasses reranking altogether. Detection recall for entity injection remains perfect but is accompanied by a false alarm rate near unity, indicating the controller treats essentially any entity-injected rewrite as suspect regardless of whether it happened to be benign this time — a reliable but blunt tripwire, not a discriminating judgment. Grounding corruption, by contrast, shows more discriminating behavior (60% recall against a 55% false alarm rate) alongside the study's largest, if not significant, point estimate.

RQ3's honest answer, at the sample size validated so far, is inconclusive-but-directionally-positive for content-level corruption, and negative for system-level degradation whose signal falls outside the verifier's feature family. We regard the corrected numbers, not the pre-correction ones, as the paper's reportable result, and flag the larger-scale replication explicitly as ongoing work rather than retroactively strengthening the claim.

VIII. DISCUSSION

Read together, RQ1–RQ3 sketch a coherent picture rather than three independent findings. The weak general-purpose detection ceiling from RQ1 ($r \approx 0.13$–0.15) is not simply a modeling shortfall; it appears to be near the limit of what this telemetry vocabulary can extract from this corpus, having survived three feature families and two model classes. RQ2 shows that a closed control loop built on top of that weak signal does not, on its own, buy anything on ordinary queries — the additional REPLAN attempts a calibrated controller is willing to take are themselves close to a coin flip, so the loop adds cost without adding value in the common case. RQ3's corrected result is the most interesting because it is the most honest: it neither confirms nor cleanly refutes the hypothesis that a weak detector still functions as a safety net against large, obvious failures. The point estimates stay positive for content corruption and the qualitative asymmetry between content-level and system-level fault detectability persists after correction, but the statistical case at $n = 149$ is not yet closed.

The implementation defect of Section VII-A is, in our view, as important a finding as any of the three numbered ones. A component computed purely for observability — a reranker score, a disagreement metric, any telemetry signal — is architecturally easy to leave disconnected from the outcome it is meant to explain, and that disconnection will not raise an exception; it will simply produce results that look coherent until someone asks why a specific, otherwise well-motivated intervention shows no effect. We would not have found it without deliberately investigating an anomalously weak result rather than accepting it, which we offer as a general methodological point for observability-guided systems research: a surprisingly clean or surprisingly null sub-result is a prompt to investigate the measurement, not just the hypothesis.

IX. Limitations

The recovery study's headline numbers (Table III) are reported at $n = 149$; a full test-set-scale ($n = 1175$) replication was launched concurrently with the writing of this paper and its result should be treated as the authoritative one once available, superseding the $n = 149$ figures reported here if it changes their significance in either direction. The generator is a locally hosted, quantized 2–3B parameter model under a strict two-call budget, chosen for a memory-constrained deployment target rather than for maximum capability; we do not know how these findings transfer to a larger or API-hosted generator. All reported results concern TripClick's TAIL split specifically; HEAD and TORSO results, and results on MS MARCO Chameleons and TREC DL 2019/2020, are left to future work. The verifier calibration methodology itself inherits the usual caveats of small-effect correlational analysis: several sub-experiments (Table I's footnote) demonstrate that these correlations are sensitive to sample size below a few hundred episodes, and we would not be surprised if a substantially larger calibration sample moved the reported ceiling somewhat, though we would be surprised if it moved qualitatively.

X. Conclusion

We built a closed-loop, observability-guided agent for LLM query expansion and evaluated it rigorously, at full test-set scale, against a matched static comparator, on a real long-tail search benchmark. The honest result is not that the agent works: on natural queries it does not significantly outperform a much simpler static pipeline, and it costs more. What we can report with more confidence is a precise map of *why*: qrel-free telemetry carries a real but weak general signal that a rigorous search across features and models could not meaningfully improve, and a closed control loop built on a weak verifier inherits that weakness on ordinary queries. Whether that same weak verifier functions as a safety net specifically against large, injected failures — as opposed to subtle natural variation — remains a directionally positive but not yet statistically settled question, and one we consider more promising than the paper's other results, since a detector need not be accurate in general to be useful as a tripwire against catastrophic failure specifically. We also report, in full, an implementation defect discovered and corrected mid-study, in the belief that an honest account of how such defects surface and get found is itself useful to other observability-guided systems work, not merely a footnote to a clean final number.

Acknowledgment

The authors thank the TripClick maintainers for access to the dataset under its non-commercial research license.

References

[1] L. Wang, N. Yang, and F. Wei, "Query2doc: Query expansion with large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: https://aclanthology.org/2023.emnlp-main.585/

[2] L. Zhang, Y. Wu, Q. Yang, J. Lin *et al.*, "Exploring the best practices of query expansion with large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.103/

[3] Google, "The site reliability workbook: Defining SLO," 2026. [Online]. Available: https://sre.google/sre-book/service-level-objectives/

[4] OpenTelemetry Authors, "OpenTelemetry signals," 2026. [Online]. Available: https://opentelemetry.io/docs/concepts/signals/

[5] ——, "OpenTelemetry GenAI semantic conventions 1.44.0," 2026. [Online]. Available: https://opentelemetry.io/docs/specs/semconv/

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022. [Online]. Available: https://arxiv.org/abs/2210.03629

[7] N. Rekabsaz, O. Lesota, M. Schedl, J. Brassey, and C. Eickhoff, "TripClick: The log files of a large health web search engine," 2021. [Online]. Available: https://tripdatabase.github.io/tripclick/

[8] Microsoft Research, "MS MARCO chameleons: Challenging the MS MARCO leaderboard with extremely obstinate queries," 2026. [Online]. Available: https://www.microsoft.com/en-us/research/publication/ms-marco-chameleons-challenging-the-ms-marco-leaderboard-with-extremely-obstinat

[9] NIST, "Text REtrieval conference (TREC) 2019 deep learning track," 2019. [Online]. Available: https://trec.nist.gov/data/deep2019.html

[10] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://github.com/beir-cellar/beir

[11] Hugging Face, "cross-encoder/ms-marco-MiniLM-L6-v2," 2026. [Online]. Available: https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2